



Financial Risk Intelligence Engine

Purpose

This document is the implementation roadmap for the production-grade ML Engineering/MLOps project.

Project Goal

Build a production-ready ML system that:

1. Calculates a transaction risk score.
2. Predicts whether a transaction is fraudulent/anomalous.
3. Explains WHY the prediction was made.
4. Tracks experiments, versions data/models, monitors drift, and retrains automatically.

Dataset: IEEE-CIS Fraud Detection (primary dataset)

Technology Stack: Python, Scikit-learn, XGBoost/LightGBM, FastAPI, PostgreSQL, Redis, DVC, MLflow, Airflow, Docker, Evidently AI, GitHub Actions, Terraform, Kubernetes, AWS

Architecture Decisions

- Use IEEE-CIS only for V1.
 - Model may use all features (including anonymized V/C/D features).
 - Business explanations MUST only use interpretable and engineered features.
 - Prediction and explanation are separate components.
 - Champion/Challenger model promotion.
 - Data drift monitoring and automated retraining.
-

Implementation Roadmap

Phase 1 – Repository Setup

- ☒ ~~Create repository~~
- ☒ ~~Configure virtual environment~~
- ☒ ~~Configure linting/testing/logging~~
- ☒ ~~Create folder structure~~

Phase 2 – Dataset

- ☒ ~~Download IEEE-CIS~~
- ☒ ~~Study schema~~
- ☒ ~~Load into PostgreSQL~~
- ☒ ~~Create data dictionary~~

Phase 3 – EDA

- ☒ ~~Missing values~~
- ☒ ~~Class imbalance~~
- ☒ ~~Feature distributions~~
- ☒ ~~Correlation study~~
- ☒ ~~Leakage analysis~~

Phase 4 – Data Validation and Cleaning

- ☐ Schema validation
- ☐ Missing-value checks and handle
- ☐ Data quality reports

Phase 5 – Feature Engineering

- ☐ Time features
- ☐ Transaction amount features
- ☐ Device features
- ☐ Email features
- ☐ Historical behavioral aggregates
- ☐ Risk-oriented engineered features

Phase 6 – Baseline Model

- ☐ Train XGBoost/LightGBM
- ☐ Evaluate ROC-AUC, PR-AUC, F1
- ☐ Calibrate probabilities

Phase 7 – Explainability

- ☐ SHAP
- ☐ Analyst explanation layer
- ☐ Debug explanation layer
- ☐ Template-based business reasons

Phase 8 – Experiment Tracking

- ☐ MLflow
- ☐ Hyperparameter logging
- ☐ Model Registry

Phase 9 – Data Versioning

- ☐ DVC

- ☐ Version raw/intermediate/final datasets

Phase 10 – API

- ☐ FastAPI
- ☐ /predict endpoint
- ☐ /explain endpoint
- ☐ Validation
- ☐ Logging

Phase 11 – Feature Storage

- ☐ PostgreSQL for history
- ☐ Redis cache for low-latency serving

Phase 12 – Airflow

- ☐ DAG: Validate → Feature Engineering → Train → Evaluate → Register

Phase 13 – Monitoring

- ☐ Evidently AI
- ☐ Data drift
- ☐ Prediction monitoring

Phase 14 – Retraining

- ☐ Trigger on drift
- ☐ Retrain
- ☐ Champion/Challenger comparison
- ☐ Promote if improved

Phase 15 – Containerization

- ☐ Docker
- ☐ Docker Compose

Phase 16 – Infrastructure

- ☐ Terraform
- ☐ AWS S3
- ☐ AWS ECR
- ☐ AWS RDS
- ☐ IAM

Phase 17 – Deployment

- ☐ Kubernetes (kind/minikube)
- ☐ Optional AWS deployment

Phase 18 – CI/CD

- ☐ GitHub Actions
- ☐ Test
- ☐ Build
- ☐ Push image
- ☐ Deploy

Phase 19 – Documentation

- ☐ README
- ☐ Architecture diagrams
- ☐ API docs
- ☐ Interview notes

Success Criteria

The project is complete when it demonstrates

- Production ML architecture
- Explainable AI • Automated ML pipeline
- Reproducibility • Monitoring
- Drift detection
- Automated retraining

- Deployment
- Infrastructure as Code
- Interview-ready documentation

File Structure of the Project

```
Adaptive-Financial-Risk-Intelligence-Engine/
```

```
|— .github/  
|— airflow/  
|— api/  
|— apps/  
|— configs/  
|— data/  
|— database/  
|— deployment/  
|— docker/  
|— docs/  
|— dvc/  
|— explainability/  
|— feature_store  
|— infrastructure/  
|— ml/  
|— monitoring/  
|— pipelines/  
|— scripts/  
|— shared/  
|— tests/  
|— notebooks/  
|— docker-compose.yml  
|— pyproject.toml  
|— requirements.txt  
|— README.md  
|— Makefile
```

```
# Adaptive-Financial-Risk-Intelligence-Engine
```

```
## .github/
```

Contains GitHub-specific configurations.

```
...
```

```
.github/
```

```
|
|— workflows/
|   |— ci.yml
|   |— cd.yml
|   └— release.yml
|
|— ISSUE_TEMPLATE/
|
|— PULL_REQUEST_TEMPLATE.md
|
└— CODEOWNERS
...
```

Purpose

- * GitHub Actions
- * CI/CD
- * Issue templates
- * Pull request templates

```
---
```

```
# airflow/
```

Everything related to Airflow.

```
...
```

```
airflow/
```

```
|
|— dags/
|   |— training_pipeline.py
|   └— retraining_pipeline.py
```

```
    └─ monitoring_pipeline.py
├─ plugins/
├─ operators/
├─ sensors/
└─ requirements.txt
`--
```

Purpose

- * Scheduling
- * Orchestration
- * Retraining

api/

Everything related to FastAPI.

api/

```
├─ routes/
│   ├── predict.py
│   ├── explain.py
│   ├── health.py
│   └─ metrics.py
├─ middleware/
├─ dependencies.py
└─ schemas.py
```

```
|— app.py
|
|— main.py
`--
`--
```

Purpose

- * REST APIs
- * Validation
- * Authentication (future)

apps/

Application startup.

apps/

```
|
|— inference_service.py
|— monitoring_service.py
|— training_service.py
|— retraining_service.py
`--
`--
```

Purpose

Entry points for different services.

configs/

Every configuration.

configs/

```
|
|— model.yaml
|— airflow.yaml
|— database.yaml
|— api.yaml
|— monitoring.yaml
|— logging.yaml
|— aws.yaml
└— paths.yaml
`--
```

Purpose

No hardcoded values anywhere.

data/

Only datasets.

data/

```
|
|— raw/
|— interim/
|— processed/
|— external/
└— validation_reports/
`--
```

Purpose

Temporary local datasets.

DVC will version these.

```
# database/
```

Database layer.

```
...
```

```
database/
```

```
|
```

```
|— connection.py
```

```
|— loader.py
```

```
|— schema.py
```

```
|— queries.py
```

```
|— session.py
```

```
|— migrations/
```

```
...
```

Purpose

Only database interaction.

No ML logic.

```
---
```

```
# deployment/
```

Deployment configuration.

```
...
```

```
deployment/
```

```
|
```

```
|— docker/
```

```
|— kubernetes/
```

```
|— aws/
```

```
|— scripts/
```

```
...
```

Purpose

Deployment assets.

docker/

Docker configuration.

\\

docker/

|

├─ Dockerfile.api

├─ Dockerfile.training

├─ Dockerfile.airflow

├─ .dockerignore

└─ entrypoint.sh

\\

Purpose

Containerization.

docs/

Documentation.

\\

docs/

|

├─ architecture.md

├─ api.md

├─ ml_pipeline.md

├─ deployment.md

├─ roadmap.md

└─ interview_notes.md


```
```
```

Purpose

Project documentation.

```

```

```
dvc/
```

DVC metadata.

```
```
```

dvc/

|

|— pipelines/

|— stages/

|— params.yaml

```
```
```

Purpose

Dataset versioning.

```

```

```
explainability/
```

Explainability layer.

```
```
```

explainability/

|

|— shap_engine.py

|— analyst_view.py

|— debug_view.py

|— feature_mapper.py

|— templates.py

```
└─ explanation_service.py
...
```

Purpose

Business explanations.

feature_store/

Feature engineering.

...

feature_store/

|

└─ feature_builder.py

└─ feature_registry.py

└─ aggregations.py

└─ validators.py

└─ metadata.py

...

Purpose

Reusable feature generation.

infrastructure/

Infrastructure as Code.

...

infrastructure/

|

└─ terraform/

└─ aws/

```
|— networking/  
|— variables/  
`--`
```

Purpose

Terraform.

ml/

Machine Learning.

`--`

ml/

|

|— training/

|— inference/

|— evaluation/

|— calibration/

|— registry/

|— models/

|— utils/

`--`

Purpose

Everything related to ML.

monitoring/

Monitoring.

`--`

monitoring/

```
|
|— evidently/
|— metrics.py
|— drift.py
|— alerts.py
|— dashboards.py
|—
|—
```

Purpose

Model monitoring.

pipelines/

ML pipelines.

pipelines/

```
|
|— ingestion_pipeline.py
|— validation_pipeline.py
|— cleaning_pipeline.py
|— feature_engineering_pipeline.py
|— dataset_builder_pipeline.py
|— training_pipeline.py
|— evaluation_pipeline.py
|— calibration_pipeline.py
|— registration_pipeline.py
|— inference_pipeline.py
|— explainability_pipeline.py
|— monitoring_pipeline.py
|— retraining_pipeline.py
|— deployment_pipeline.py
```

Purpose

End-to-end ML workflow.

```
---

# scripts/

Helper scripts.

...

scripts/
|
|— download_dataset.py
|— seed_database.py
|— train.py
|— predict.py
|— explain.py
|— cleanup.py
...

Purpose

CLI utilities.

---

# shared/

Shared utilities.

...

shared/
|
|— logger.py
|— constants.py
|— exceptions.py
|— decorators.py
|— helpers.py
|— enums.py
...
```

Purpose

Shared code used everywhere.

tests/

Testing.

\\

tests/

|

|— unit/

|— integration/

|— api/

|— ml/

|— performance/

\\

Purpose

Testing.

notebooks/

Research only.

\\

notebooks/

|

|— 01_eda.ipynb

|— 02_feature_engineering.ipynb

|— 03_model_experiments.ipynb

|— archive/

```

'''

Purpose

Exploration.

No production code should live here.

---

# Root Files

## docker-compose.yml

Runs:

* PostgreSQL
* Redis
* MLflow
* Airflow
* API

---

## pyproject.toml

Python project configuration.

---

## requirements.txt

Python dependencies.

---

## README.md
```

Main project documentation.

Makefile

Development shortcuts.

Examples:

```

make train

make api

make airflow

make test

make lint

make deploy

```

Database information

- **Database name:** fraud_risk
- **user:** fraud_user
- **password:** admin
- **Privileges:** sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE fraud_risk TO fraud_user;"

Project Setup

1. Set Up pyproject.toml – bold text is command

1. **poetry init** – create the pyproject.toml file automatically
2. Adding dependencies – **poetry add pandas scikit-learn dvc**
3. **poetry lock** – creates the poetry.lock file for reproducibility
4. **poetry install** - Install everything
5. Export requirements.txt to toml file: **poetry export -f requirements.txt --output requirements.txt**

2. About Dataset

1. IEEE-CIS Fraud Detection: [Link](#) | [Kaggle](#)

2. Dataset Description

- a. In this competition you are predicting the probability that an online transaction is fraudulent, as denoted by the binary target **isFraud**.
- b. The data is broken into two files identity and transaction, which are joined by TransactionID. Not all transactions have corresponding identity information.

c. Categorical Features - Transaction

- i. ProductCD
 - ii. card1 - card6
- addr1, addr2
1. P_emaildomain
 2. R_emaildomain
 3. M1 - M9

4. Categorical Features - Identity

- a. DeviceType
 - b. DeviceInfo
- id_12 - id_38
1. The TransactionDT feature is a timedelta from a given reference datetime (not an actual timestamp).
 2. You can read more about the data from [this post by the competition host](#).

3. Files

- a. **train_{transaction, identity}.csv** the training set
- b. **test_{transaction, identity}.csv** the test set (you must predict the isFraud value for these observations)
- **sample_submission.csv** a sample submission file in the correct format

Coding Step Need to do - **IMPORTANT**

1. **For Group Structure**, as having multiple transactions from a single customer might introduce the **Bias in the model**, hence, we will use **Group K-Fold to tackle this problem**.
2. Keep the eye on the StratifiedGroupKFold in sklearn for imbalance datasets.
3. Prioritize **Recall** (catch as many frauds as possible).
4. Use **F β -score** ($\beta > 1$ gives more weight to Recall).
5. Cost sensitive learning - class weights

Tier 1 - Candidate for model training

Feature	Diverges by class?	Play part in model training	Note
TransactionAmt	Yes	Yes	Right-skewed; log-transform; strong candidate for deviation-from-baseline feature
dist1	Yes	Yes (caveat)	~94% missing — structural; needs missing-indicator, not imputation
dist2	Yes	Yes (caveat)	Same missingness caveat as dist1
card1	Weak	No	High cardinality identifier; useful only for synthetic identity key
card2	Weak	No	~500 unique values, no ordinal meaning
card3	Weak	No	~90% concentrated in 2 values — low variance

Feature	Diverges by class?	Play part in model training	Note
card5	Weak	No	Similar concentration issue to card3
card4	Yes	Yes	discover/mastercard/visa show different fraud rates
card6	Yes	Yes	credit >> debit fraud rate
ProductCD	Yes	Yes (strong)	C code disproportionately fraud vs. W
P_emaildomain	Yes	Yes (strong)	protonmail.com extreme outlier signal
R_emaildomain	Yes	Yes	Similar pattern to P_emaildomain, weaker
DeviceType	Yes	Yes	mobile ~10.2% vs desktop ~6.5% fraud rate
DeviceInfo	Yes	Yes (predictive)	Windows/iOS/MacOS provide broad coverage but may not separate fraud strongly.
addr1	Weak	Yes (missingness)	Addr1 is weak as a raw categorical feature, but strong as a missingness indicator . The fact that certain device builds have disproportionately high fraud rates means DeviceInfo is predictive - but only if handled carefully.
addr2	Weak	Yes	addr2 is highly predictive , despite skew, because certain bins strongly correlate with fraud.
M1	No	No	Almost all "T"; F has only 25 samples
M2	Yes	Yes	F ~2x fraud rate of T (3.49% vs 1.81%)
M3	Yes	Yes	F higher than T (3.03% vs 1.71%)
M4	Yes	Yes (strong)	Clear separation across categories
M5	Yes	Yes	T higher fraud rate than F

Feature	Diverges by class?	Play part in model training	Note
M6	Weak	Yes	F has higher fraud rate than T, Candidate, but lower priority.
M7	No	No	Low variance + weak separation → not very predictive.
M8	Yes	Yes	Moderate variance, and here the fraud rate differs more noticeably. Could add signal.
M9	Yes	Yes (strong)	Stronger separation in fraud rate. Even though imbalanced, this feature looks predictive.

Feature drop

1. What to Drop (The "Useless" Features)

Features with an IV < 0.02 add noise to your model without adding predictive power.

- **Drop the raw values:** `card4`, `M1`, `M7`, `M8`, `R_emaildomain` (due to leakage of Fraud rate 0.0 in all domain)
- `c3`: IV is 0.0000 (useless).
- `id_10`, `id_11`: IV is 0.0000 (useless).
- `id_12`, `id_13`, `id_37`: IVs are ~0.01 (useless).

2. What to Keep (The Strong & Medium Signals)

These are your core predictors. They have proven mathematical separation between fraud and non-fraud classes without crossing the 0.5 leakage threshold.

- **Strong (0.3 to 0.5):** `card3`, `M4`, `ProductCD`
- **Medium (0.1 to 0.3):** `TransactionAmt`, `card6`, `P_emaildomain`, `DeviceInfo`.
- **Weak but useful (0.02 to 0.1):** `card1`, `card2`, `card5`, `dist1`, `dist2`, `DeviceType`, `M2`, `M3`, `M5`, `M6`, `M9`. (Keep these for now; tree-based models can often extract non-linear interactions from weak individual features).

- **The `addr` Anomaly:** The IV for `addr1` (0.0111) and `addr2` (0.0107) marks them as useless. However, your Section 5 notes are incredibly sharp here: "Addr1 is weak as a raw categorical feature, but strong as a missingness indicator". You correctly identified that NaNs are disproportionately high in fraud cases (~37%). **Do not drop them entirely.** Keep them specifically to extract a binary `is_missing` flag, or fill NaNs with `999`, but do not expect the raw address codes to help the model.

3. What to Keep in C, V , id, D features

in V feature

['V232', 'V129', 'V43', 'V276', 'V182', 'V99', 'V328', 'V287', 'V52', 'V156', 'V277', 'V256', 'V251', 'V150', 'V237', 'V249', 'V244', 'V89', 'V271', 'V191', 'V222', 'V10', 'V48', 'V292', 'V302', 'V69', 'V187', 'V90', 'V29', 'V74', 'V199', 'V158', 'V309', 'V162', 'V81', 'V59', 'V225', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V12', 'V13', 'V14', 'V19', 'V20', 'V23', 'V24', 'V25', 'V26', 'V35', 'V36', 'V37', 'V38', 'V39', 'V40', 'V41', 'V44', 'V45', 'V46', 'V47', 'V53', 'V54', 'V55', 'V56', 'V61', 'V62', 'V64', 'V65', 'V66', 'V67', 'V68', 'V75', 'V76', 'V77', 'V78', 'V82', 'V83', 'V85', 'V86', 'V87', 'V88', 'V98', 'V107', 'V108', 'V109', 'V110', 'V111', 'V112', 'V113', 'V114', 'V115', 'V116', 'V117', 'V118', 'V119', 'V120', 'V121', 'V122', 'V123', 'V124', 'V125', 'V130', 'V131', 'V135', 'V136', 'V137', 'V138', 'V139', 'V140', 'V141', 'V142', 'V146', 'V147', 'V152', 'V161', 'V165', 'V166', 'V169', 'V170', 'V171', 'V172', 'V173', 'V174', 'V175', 'V176', 'V181', 'V183', 'V184', 'V185', 'V186', 'V188', 'V189', 'V194', 'V195', 'V197', 'V198', 'V200', 'V201', 'V207', 'V208', 'V209', 'V210', 'V220', 'V223', 'V226', 'V227', 'V228', 'V229', 'V230', 'V235', 'V238', 'V239', 'V240', 'V241', 'V243', 'V245', 'V246', 'V247', 'V252', 'V257', 'V258', 'V259', 'V260', 'V261', 'V262', 'V264', 'V267', 'V268', 'V274', 'V281', 'V282', 'V283', 'V284', 'V286', 'V288', 'V289', 'V290', 'V300', 'V301', 'V303', 'V305', 'V310', 'V312', 'V313', 'V314', 'V315', 'V319', 'V320', 'V321', 'V325', 'V335', 'V336', 'V337', 'V338', 'V339']

in C feature

['C1', 'C2', 'C4', 'C5', 'C6', 'C7', 'C8', 'C9', 'C10', 'C11', 'C12', 'C13', 'C14']

in id Feature

['id_01', 'id_02', 'id_03', 'id_04', 'id_05', 'id_06', 'id_07', 'id_08', 'id_09', 'id_10', 'id_11', 'id_12', 'id_13', 'id_14', 'id_15', 'id_16', 'id_17', 'id_18', 'id_19', 'id_20', 'id_21']

```
'id_22', 'id_23', 'id_24', 'id_25', 'id_26', 'id_27', 'id_28', 'id_29', 'id_30', 'id_31',  
'id_32', 'id_33', 'id_34', 'id_35', 'id_36', 'id_37', 'id_38']
```

in D feature

```
['D1', 'D2', 'D3', 'D4', 'D5', 'D6', 'D7', 'D8', 'D9', 'D10', 'D11', 'D12', 'D13', 'D14',  
'D15']
```

Logging folder structure

```
logs/  
|  
├─ application.log  
├─ api.log  
├─ training.log  
├─ airflow.log  
└─ monitoring.log
```

this we'll develop as we code this not the

Pipeline Folder Structure

```
pipelines/  
|  
├─ ingestion_pipeline.py  
├─ validation_pipeline.py  
├─ cleaning_pipeline.py  
├─ feature_engineering_pipeline.py  
├─ dataset_builder_pipeline.py  
├─ training_pipeline.py  
├─ evaluation_pipeline.py  
├─ calibration_pipeline.py  
├─ registration_pipeline.py  
├─ inference_pipeline.py  
├─ explainability_pipeline.py  
└─ monitoring_pipeline.py
```

```
|— retraining_pipeline.py
|— deployment_pipeline.py
```

I want each pipeline to have **exactly one responsibility**.

That makes Airflow integration much easier later because each file can become one Airflow task.

Notice

These are **pipelines**, not utility classes.

Each one represents a stage of the ML lifecycle.

1. ingestion_pipeline.py

Responsibility

Move data from the outside world into our system.

Workflow

```
CSV
↓
Database Loader
↓
PostgreSQL
```

Uses

- Database
- CSVLoader

Output

```
transactions_raw
identity_raw
```

2. validation_pipeline.py

Responsibility

Check if the raw data is usable.

Checks

- Missing values
- Duplicate TransactionID
- Wrong data types
- Null percentage
- Invalid categories
- Schema consistency

Output

```
validation_report.json
```

If validation fails

Stop the pipeline.

3. cleaning_pipeline.py

Responsibility

Clean the validated data.

Examples

- Fill missing values
- Convert dtypes
- Remove impossible rows
- Normalize categorical values

Output

```
cleaned.parquet
```

Database remains unchanged.

4. feature_engineering_pipeline.py

Responsibility

Create ML features.

Examples

- transaction_hour
- transaction_day
- email_match
- log_transaction_amount
- frequency features
- behavioural aggregates

Output

```
features.parquet
```

5. dataset_builder_pipeline.py

Responsibility

Prepare the final ML dataset.

Examples

Split

```
x  
y
```

Train

Validation

Test

Apply

- scaling (if required)
- encoding
- sampling strategy

Output

```
train.parquet  
validation.parquet  
test.parquet
```

6. training_pipeline.py

Responsibility

Train models.

Tasks

- Read train.parquet
- Train XGBoost
- Hyperparameter tuning
- Save model
- Log MLflow experiment

Output

```
model.pkl
```

7. evaluation_pipeline.py

Responsibility

Evaluate the model.

Metrics

- ROC-AUC
- Precision
- Recall
- PR-AUC
- F1

- Confusion Matrix

Output

```
evaluation.json
```

8. calibration_pipeline.py

Responsibility

Calibrate probabilities.

Remember

Risk Score $\neq$ Raw XGBoost probability.

We'll calibrate using

- Platt Scaling
- Isotonic Regression

Output

```
calibrated_model.pkl
```

9. registration_pipeline.py

Responsibility

Model Registry.

Tasks

- Compare Champion
- Compare Challenger
- Register model in MLflow
- Promote if better

Output

MLflow Registry

10. inference_pipeline.py

Responsibility

Prediction.

Input

```
New transaction
↓
Feature engineering
↓
Model
↓
Risk Score
```

Output

Prediction object.

11. explainability_pipeline.py

Responsibility

Generate explanations.

Tasks

- SHAP
- Analyst View
- Debug View
- Feature Mapping
- Business explanation

Output

```
{
  risk_score

  reasons

  confidence
```

```
model_version
}
```

12. monitoring_pipeline.py

Responsibility

Monitor production.

Checks

- Data Drift
- Prediction Drift
- Missing values
- Feature Distribution
- Model metrics

Uses

Evidently.

13. retraining_pipeline.py

Responsibility

Retrain model.

Trigger

- Scheduled

or

- Drift

Workflow

New Data

↓

Train

↓

Evaluate

↓
Register
↓
Deploy

14. deployment_pipeline.py

Responsibility

Deploy.

Tasks

- Build Docker
- Push Image
- Deploy API
- Update Kubernetes

Pipeline dependency

Ingestion
↓
Validation
↓
Cleaning
↓
Feature Engineering
↓
Dataset Builder
↓
Training
↓

Evaluation

↓

Calibration

↓

Registration

↓

Deployment

↓

Inference

↓

Explainability

↓

Monitoring

↓

Retraining

One thing I would improve

I will create a **base pipeline**.

```
pipelines/  
base_pipeline.py
```

Every other pipeline would inherit from it.

Not because inheritance is required, but because every pipeline will eventually perform similar operations:

- logging
- configuration loading
- timing

- exception handling
- artifact tracking
- status reporting

Instead of duplicating that logic in 14 files, `BasePipeline` can provide a common lifecycle. Then each concrete pipeline only implements its own business logic.